

Assignment-13.3

Name:B.Akshara

Ht No:2303A51279

Batch:05

Task-01:

Prompt:

Objective

To eliminate repeated logic by extracting reusable functions. Refactor a legacy Python script that contains repeated blocks of code calculating the area and perimeter of rectangles.

Starter (Legacy) Code

```
# Legacy script with repeated logic
```

```
print("Area of Rectangle:", 5 * 10)
```

```
print("Perimeter of Rectangle:", 2 * (5 + 10))
```

```
print("Area of Rectangle:", 7 * 12)
```

```
print("Perimeter of Rectangle:", 2 * (7 + 12))
```

```
print("Area of Rectangle:", 10 * 15)
```

```
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Expected Outcome

- A reusable function to calculate area and perimeter
- No duplicated code blocks
- Proper docstrings for all function

Code:

```
def calculate_area(length, width):
```

```
    """Calculate the area of a rectangle.
```

```
    Args:
```

```
        length (float): The length of the rectangle.
```

```
        width (float): The width of the rectangle.
```

```
    Returns:
```

```
        float: The area of the rectangle.
```

```
    """
```

```
return length * width
```

```
def calculate_perimeter(length, width):
```

```
    """Calculate the perimeter of a rectangle.
```

```
    Args:
```

```
        length (float): The length of the rectangle.
```

```
        width (float): The width of the rectangle.
```

```
    Returns:
```

```
        float: The perimeter of the rectangle.
```

```
    """
```

```
    return 2 * (length + width)
```

```
# Example usage
```

```
rectangles = [(5, 10), (7, 12), (10, 15)]
```

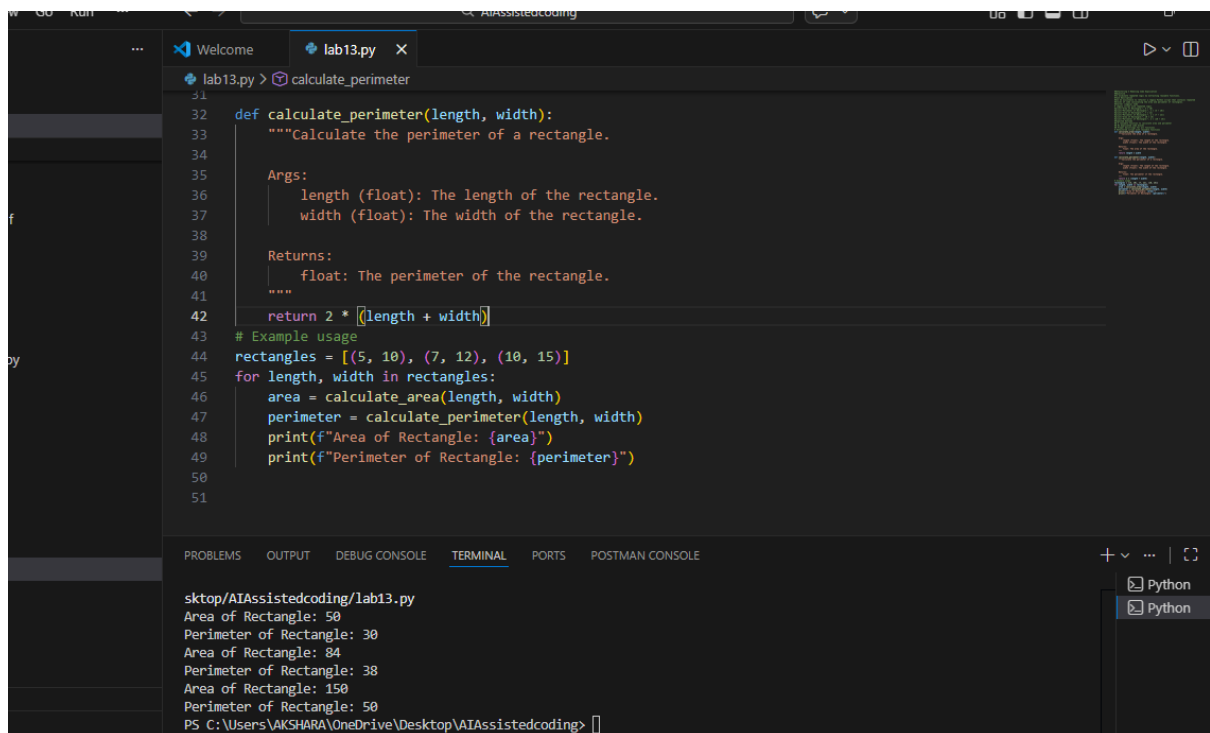
```
for length, width in rectangles:
```

```
    area = calculate_area(length, width)
```

```
    perimeter = calculate_perimeter(length, width)
```

```
    print(f'Area of Rectangle: {area}')
```

```
    print(f'Perimeter of Rectangle: {perimeter}')
```



The screenshot shows a Visual Studio Code editor with a Python file named `lab13.py`. The code defines a `calculate_perimeter` function and uses it in a loop to calculate the area and perimeter for three rectangles. The terminal output shows the results of the calculations.

```
def calculate_perimeter(length, width):
    """Calculate the perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The perimeter of the rectangle.
    """
    return 2 * (length + width)

# Example usage
rectangles = [(5, 10), (7, 12), (10, 15)]
for length, width in rectangles:
    area = calculate_area(length, width)
    perimeter = calculate_perimeter(length, width)
    print(f'Area of Rectangle: {area}')
    print(f'Perimeter of Rectangle: {perimeter}')
```

Terminal Output:

```
sktop/AIAssistedcoding/lab13.py
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
Perimeter of Rectangle: 50
PS C:\Users\AKSHARA\OneDrive\Desktop\AIAssistedcoding>
```

Explanation:

In the refactored script, we have created two functions: `calculate area` and `calculate perimeter`. These functions take the length and width of a rectangle as arguments and return the area and perimeter, respectively. This eliminates the need for repeated code blocks, making the script cleaner and more maintainable. Each function also includes a docstring that explains its purpose, parameters, and return value.

Task-02:**Prompt:**

To improve performance by replacing inefficient nested loops with optimized structures.

analyze and refactor a script that checks the presence of elements using nested loops.

Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f'{s} is in the list')
    else:
        print(f'{s} is not in the list')
```

Expected Outcome

- Optimized solution using set lookups or comprehensions
- Performance comparison before and after refactoring

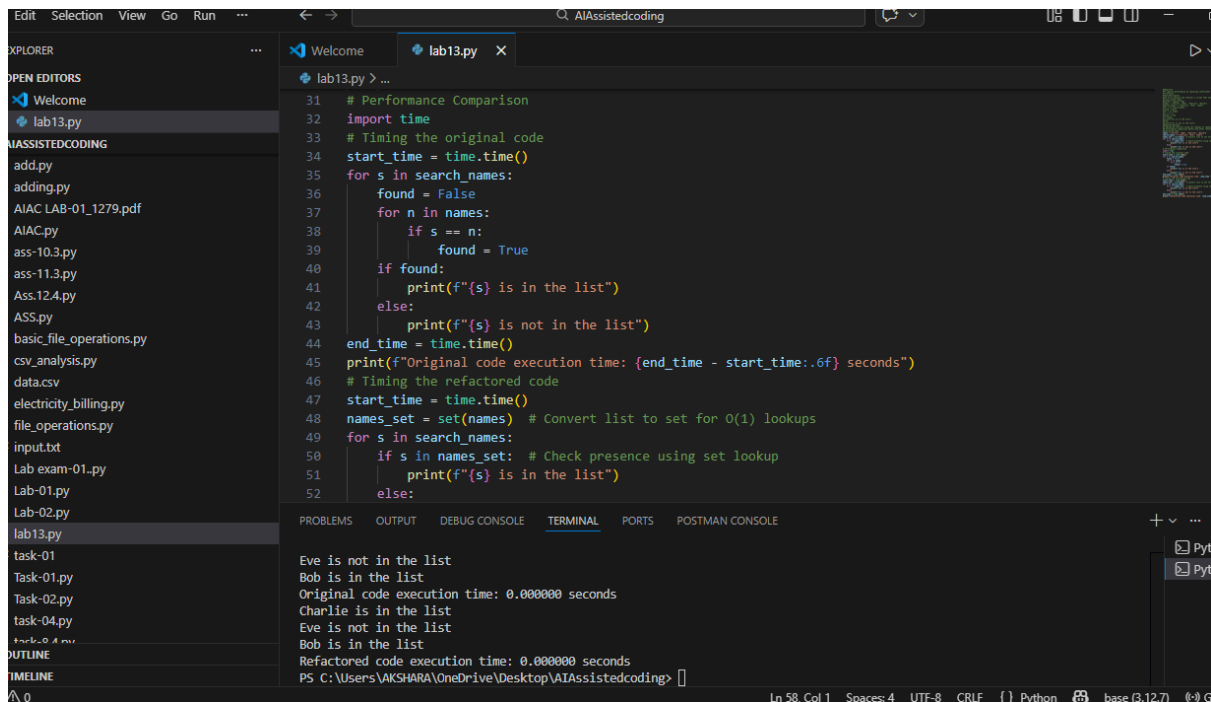
Code:

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
names_set = set(names) # Convert list to set for O(1) lookups
for s in search_names:
```

```

    if s in names_set: # Check presence using set lookup
        print(f'{s} is in the list')
    else:
        print(f'{s} is not in the list')
# Performance Comparison
import time
# Timing the original code
start_time = time.time()
for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f'{s} is in the list')
    else:
        print(f'{s} is not in the list')
end_time = time.time()
print(f'Original code execution time: {end_time - start_time:.6f} seconds')
# Timing the refactored code
start_time = time.time()
names_set = set(names) # Convert list to set for O(1) lookups
for s in search_names:
    if s in names_set: # Check presence using set lookup
        print(f'{s} is in the list')
    else:
        print(f'{s} is not in the list')
end_time = time.time()
print(f'Refactored code execution time: {end_time - start_time:.6f} seconds')

```



```
31 # Performance Comparison
32 import time
33 # Timing the original code
34 start_time = time.time()
35 for s in search_names:
36     found = False
37     for n in names:
38         if s == n:
39             found = True
40     if found:
41         print(f"{s} is in the list")
42     else:
43         print(f"{s} is not in the list")
44 end_time = time.time()
45 print(f"Original code execution time: {end_time - start_time:.6f} seconds")
46 # Timing the refactored code
47 start_time = time.time()
48 names_set = set(names) # Convert list to set for O(1) lookups
49 for s in search_names:
50     if s in names_set: # Check presence using set lookup
51         print(f"{s} is in the list")
52     else:
```

```
Problems OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
Eve is not in the list
Bob is in the list
Original code execution time: 0.000000 seconds
Charlie is in the list
Eve is not in the list
Bob is in the list
Refactored code execution time: 0.000000 seconds
PS C:\Users\AKSHARA\OneDrive\Desktop\AIAssistedcoding>
```

Explanation:

The original code uses nested loops to check for the presence of each search name in the list of names, resulting in $O(n*m)$ time complexity. The refactored code converts the list of names into a set, allowing for $O(1)$ average time complexity for lookups. This significantly improves performance, especially as the size of the lists increases. The performance comparison demonstrates the efficiency gained from using sets over nested loops.

Task-03

Prompt:

To modularize code by extracting calculations into reusable functions.

Refactor a legacy script where price and tax calculations are written inline.

Starter (Legacy) Code

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Expected Outcome

- A function `calculate_total(price)`
- Cleaner and reusable code structure
- Proper documentation

Code:

```
def calculate_total(price):  
    """  
  
    Calculate the total price including tax.  
  
    Parameters:  
  
    price (float): The original price of the item.  
  
    Returns:  
  
    float: The total price after adding tax.  
  
    """  
  
    tax = price * 0.18  
    total = price + tax  
    return total  
  
# Example usage  
price1 = 250  
total1 = calculate_total(price1)  
print("Total Price for item 1:", total1)  
price2 = 500  
total2 = calculate_total(price2)  
print("Total Price for item 2:", total2)
```

```
17  ## Cleaner and reusable code structure
18  ## Proper documentation
19  def calculate_total(price):
20      """
21      Calculate the total price including tax.
22
23      Parameters:
24      price (float): The original price of the item.
25
26      Returns:
27      float: The total price after adding tax.
28      """
29      tax = price * 0.18
30      total = price + tax
31      return total
32
33  # Example usage
34  price1 = 250
35  total1 = calculate_total(price1)
36  print("Total Price for item 1:", total1)
37  price2 = 500
38  total2 = calculate_total(price2)
39  print("Total Price for item 2:", total2)
```

skttop/AIAssistedcoding/lab13.py
Total Price for item 1: 295.0
Total Price for item 2: 590.0
PS C:\Users\AKSHARA\OneDrive\Desktop\AIAssistedcoding> & C:\Users\AKSHARA\anaconda3\python.exe c:/Users/AKSHARA/OneDrive/De
skttop/AIAssistedcoding/lab13.py
Total Price for item 1: 295.0
Total Price for item 2: 590.0

Explanation:

In this refactored code, we have created a function called calculate total that takes the price as an argument and calculates the total price by adding the tax. This function can be reused for any price input, making the code cleaner and more modular. The function also includes a docstring that explains its purpose, parameters, and return value, improving code readability and maintainability.

Task-04:

Prompt:

To improve maintainability by replacing magic numbers with named constants.

Identify hardcoded values and replace them with constants.

Starter (Legacy) Code

```
print("Area of Circle:", 3.14159 * (7 ** 2))
```

```
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Expected Outcome

- Constants such as PI and RADIUS
- Cleaner, maintainable code

Code:

```
PI = 3.14159
```

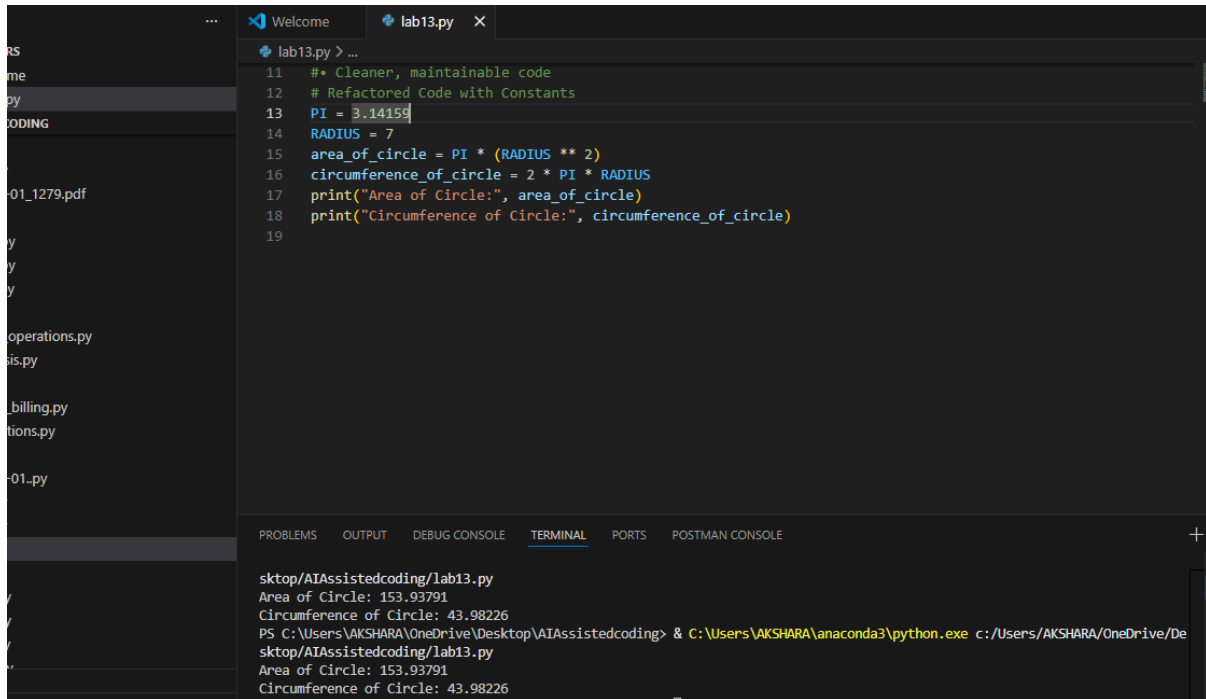
```
RADIUS = 7
```

```
area_of_circle = PI * (RADIUS ** 2)
```

```
circumference_of_circle = 2 * PI * RADIUS
```

```
print("Area of Circle:", area_of_circle)
```

```
print("Circumference of Circle:", circumference_of_circle)
```



```
lab13.py > ...
11  /* Cleaner, maintainable code
12  # Refactored Code with Constants
13  PI = 3.14159
14  RADIUS = 7
15  area_of_circle = PI * (RADIUS ** 2)
16  circumference_of_circle = 2 * PI * RADIUS
17  print("Area of Circle:", area_of_circle)
18  print("Circumference of Circle:", circumference_of_circle)
19

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  POSTMAN CONSOLE

skttop/AIAssistedcoding/lab13.py
Area of Circle: 153.93791
Circumference of Circle: 43.98226
PS C:\Users\AKSHARA\OneDrive\Desktop\AIAssistedcoding> & C:\Users\AKSHARA\anaconda3\python.exe c:/Users/AKSHARA/OneDrive/De
skttop/AIAssistedcoding/lab13.py
Area of Circle: 153.93791
Circumference of Circle: 43.98226
```

Explanation:

In the refactored code, we defined constants `PI` and `RADIUS` to replace the hardcoded values. This makes the code more readable and maintainable, as we can easily update the values in one place if needed.

Task-05

Prompt:

To enhance readability using descriptive variable names and comments.

Refactor a script with unclear variable names.

Starter (Legacy) Code

```
a = 10
```

```
b = 20
```

```
c = a * b / 2
```

```
print(c)
```

Expected Outcome

- Descriptive variable names

Inline comments explaining logic

- Identical output

Code:

#Refactored Code

```
# Calculate the area of a triangle using base and height

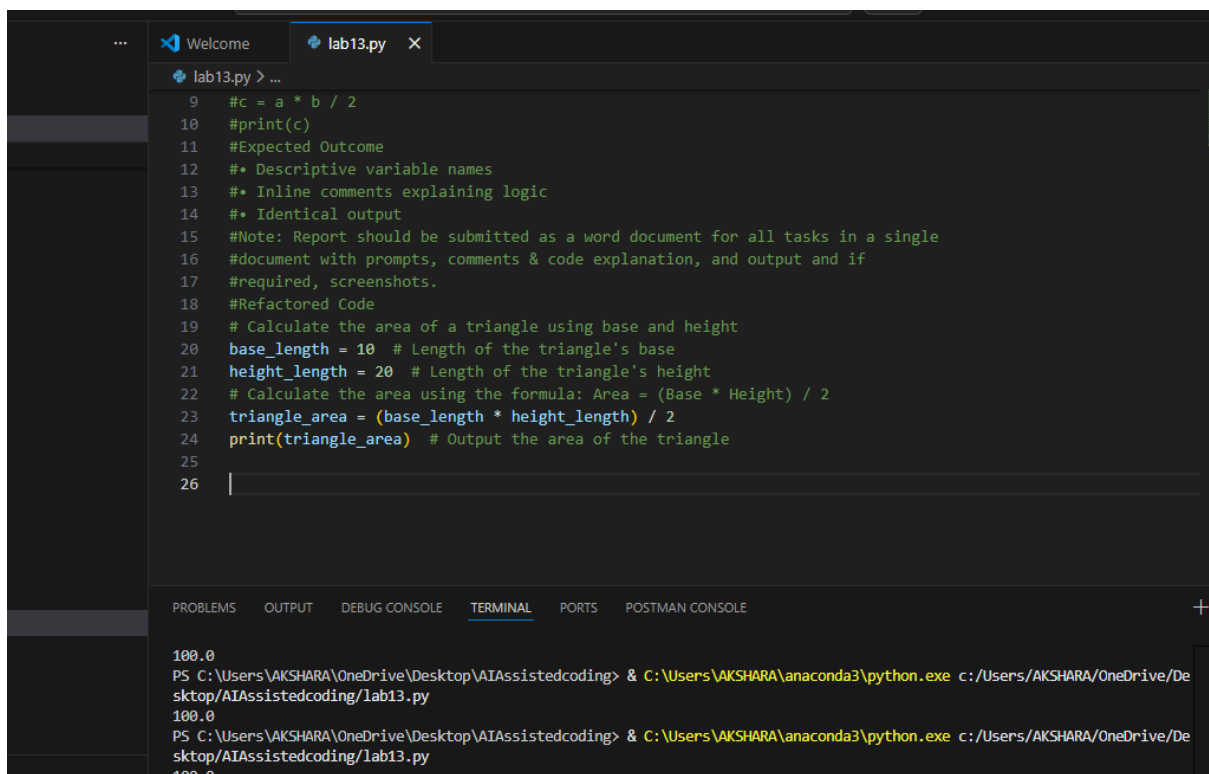
base_length = 10 # Length of the triangle's base

height_length = 20 # Length of the triangle's height

# Calculate the area using the formula: Area = (Base * Height) / 2

triangle_area = (base_length * height_length) / 2

print(triangle_area) # Output the area of the triangle
```



```
9 #c = a * b / 2
10 #print(c)
11 #Expected Outcome
12 #* Descriptive variable names
13 #* Inline comments explaining logic
14 #* Identical output
15 #Note: Report should be submitted as a word document for all tasks in a single
16 #document with prompts, comments & code explanation, and output and if
17 #required, screenshots.
18 #Refactored Code
19 # Calculate the area of a triangle using base and height
20 base_length = 10 # Length of the triangle's base
21 height_length = 20 # Length of the triangle's height
22 # Calculate the area using the formula: Area = (Base * Height) / 2
23 triangle_area = (base_length * height_length) / 2
24 print(triangle_area) # Output the area of the triangle
25
26 |
```

100.0
PS C:\Users\AKSHARA\OneDrive\Desktop\AIAssistedcoding> & C:\Users\AKSHARA\anaconda3\python.exe c:/Users/AKSHARA/OneDrive/Desktop/AIAssistedcoding/lab13.py
100.0
PS C:\Users\AKSHARA\OneDrive\Desktop\AIAssistedcoding> & C:\Users\AKSHARA\anaconda3\python.exe c:/Users/AKSHARA/OneDrive/Desktop/AIAssistedcoding/lab13.py
100.0

Explanation:

In the refactored code, we have renamed the variables to be more descriptive of their purpose.

'a' is renamed to 'base length' to indicate that it represents the length of the triangle's base. 'b' is renamed to 'height length' to indicate that it represents the length of the triangle's height. 'c' is renamed to 'triangle area' to clearly indicate that it holds the calculated area of the triangle.